

BÁO CÁO THỰC HÀNH: ỨNG DỤNG KỸ THUẬT PROMPT ENGINEERING TRONG HỌC TẬP

I. Lựa chọn và phân tích tác vụ học tập

Để đánh giá toàn diện hiệu quả của các mô hình ngôn ngữ lớn (LLM), tôi đã lựa chọn 3 tác vụ học tập phổ biến trong khối ngành Công nghệ Thông tin, đi từ việc tiếp thu lý thuyết, hiểu sâu bản chất đến kiểm tra đánh giá.

1. Tác vụ 1: Tóm tắt tài liệu học thuật (Chủ đề: Tính đóng gói - Encapsulation trong Lập trình Hướng đối tượng Java)

- Mục tiêu:** Rút gọn lý thuyết chuyên ngành thành các gạch đầu dòng cốt lõi, giúp sinh viên ghi nhớ nhanh cách bảo vệ dữ liệu trong lập trình.
- Thách thức:** Đảm bảo AI không chỉ nêu định nghĩa suông mà phải đưa ra được cách thức triển khai thực tế (ví dụ: dùng private modifier và getter/setter).

2. Tác vụ 2: Giải thích một khái niệm phức tạp (Chủ đề: Hiện tượng Bế tắc - Deadlock trong Hệ điều hành)

- Mục tiêu:** Cụ thể hóa một trạng thái lỗi hệ thống trừu tượng thành hình ảnh trực quan, dễ hiểu.
- Thách thức:** Khái niệm phân bổ tài nguyên của CPU thường rất khô khan. Yêu cầu đặt ra là AI phải tìm được một sự so sánh (ẩn dụ) đời thường mang tính chính xác cao để minh họa.

3. Tác vụ 3: Tạo bộ câu hỏi ôn tập (Chủ đề: Nhận diện Tấn công lừa đảo - Phishing trong An toàn thông tin)

- Mục tiêu:** Xây dựng công cụ kiểm tra kiến thức thông qua các tình huống thực tế để ôn thi.
- Thách thức:** AI thường tạo ra các câu hỏi quá dễ nhận biết. Cần ép AI sinh ra các tình huống lừa đảo tinh vi, kèm theo giải thích chi tiết tại sao tình huống đó lại nguy hiểm.

II & III. Xây dựng, Thử nghiệm và So sánh các phiên bản Prompt

Dưới đây là 3 phiên bản prompt cho mỗi tác vụ và phân phân tích sắc thái kết quả sau khi tiến hành thử nghiệm thực tế.

Tác vụ 1: Tóm tắt khái niệm (Tính đóng gói trong Java)

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
Cơ bản	Tóm tắt về tính đóng gói trong Java.	AI đưa ra một đoạn văn bản định nghĩa cơ bản. Thông tin đúng nhưng thiếu cấu trúc, không làm nổi bật được cách áp dụng. Rất khó để sinh viên dùng làm tài liệu ôn tập nhanh. Tính đóng gói trong Java là một trong bốn tính chất cơ bản của lập trình hướng đối tượng, ẩn giấu thông tin bên trong của đối tượng, không cho phép các lớp trực tiếp vào dữ liệu để thay đổi trạng thái của đối tượng đó. Thay vào đó, đối dữ liệu chỉ được thực hiện thông qua các phương thức do lớp đó cung cấp, giúp bảo vệ tính toàn vẹn của dữ liệu, tránh việc dữ liệu bị thay đổi một cách tùy tiện từ bên ngoài, đồng thời giúp giấu đi các chi tiết cài đặt phức tạp, chỉ hiển thị giao diện đơn giản cho người sử dụng.
Cải tiến	Viết một bản tóm tắt chi tiết về tính đóng gói (Encapsulation) trong Java. Trình bày rõ định nghĩa và lợi ích của nó. Hãy sử dụng gạch đầu dòng (bullet point) cho dễ đọc.	Văn bản có cấu trúc rõ ràng hơn nhờ yêu cầu gạch đầu dòng. Đã nêu được định nghĩa và lợi ích. Tuy nhiên, nội dung vẫn thiếu tính thực hành, chưa chỉ ra các từ khóa kỹ thuật cần thiết. Định nghĩa: - Tính đóng gói (Encapsulation) là quá trình che giấu các chi tiết thực thi của đối tượng. - Nó gom nhóm các trạng thái (thuộc tính) và hành vi (phương thức) lại vào một thực thể duy nhất (thường là một lớp - Class). - Nguyên tắc cốt lõi là không cho phép các lớp bên ngoài can thiệp trực tiếp vào bộ phận của đối tượng. Lợi ích của tính đóng gói: Bảo vệ dữ liệu: Ngăn chặn các đối tượng khác thay đổi dữ liệu một cách tùy tiện, đảm bảo tính toàn vẹn và an toàn cho hệ thống. Dễ dàng bảo trì và nâng cấp: Khi mã nguồn bên trong một lớp thay đổi, các lớp khác sử dụng nó sẽ không bị ảnh hưởng. Tăng tính linh hoạt: Cho phép người lập trình dễ dàng giới hạn quyền truy cập như chỉ cho phép đọc hoặc chỉ cho phép ghi dữ liệu).

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
Nâng cao	Đóng vai (Role): Bạn là một Senior Java Developer đang review code cho thực tập sinh. Nhiệm vụ: Tóm tắt tính đóng gói trong OOP. Cấu trúc bắt buộc: - Định nghĩa ngắn gọn bằng ngôn ngữ đời thường. - Cách triển khai thực tế (nhắc đến private, getter/setter). - Một ví dụ về hậu quả nếu KHÔNG	Đầu ra vượt trội. AI hóa thân xuất sắc, sử dụng từ ngữ chuẩn kỹ thuật. Phần "Hậu quả nếu không dùng" đưa ra một góc nhìn phản biện rất hay (ví dụ: tuổi của user bị gán bằng số âm), giúp người học lập tức hiểu được "Tại sao phải dùng nó".

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
	dùng tính đóng gói. Giọng văn: Chuyên nghiệp, mang tính hướng dẫn.	

Chào em, qua quá trình review các module em vừa commit, anh thấy em vẫn còn chút nhầm lẫn về việc bảo vệ dữ liệu trong các Class. Để làm việc trong các dự án Java lớn, em bắt buộc phải nắm vững **Tính đóng gói (Encapsulation)**.

Dưới đây là bản tóm tắt nhanh để em hiệu chỉnh lại tư duy lập trình của mình:

1. Định nghĩa bằng ngôn ngữ đời thường

Em hãy tưởng tượng tính đóng gói giống như một **viên thuốc nhộng** hay một cái **máy pha cà phê**.

- Bên trong máy pha cà phê có đủ thứ dây điện, áp suất, nhiệt độ phức tạp (đây là **Dữ liệu** và **Logic xử lý nội bộ**).
- Nhà sản xuất không để em chạm trực tiếp vào các dây điện đó vì rất nguy hiểm và dễ làm hỏng máy.
- Thay vào đó, họ bao bọc tất cả lại và chỉ đưa ra vài cái nút: "Bật/Tắt", "Chọn loại Cafe" (đây là **Interface/Method**).

Trong lập trình, đóng gói là việc gom các thuộc tính và phương thức liên quan vào một Class, đồng thời **che giấu** những chi tiết bên trong để bảo vệ tính toàn vẹn của dữ liệu.

2. Cách triển khai thực tế trong Java

Để thực hiện đóng gói chuẩn chỉnh trong Java, chúng ta thường tuân theo quy trình "Che giấu - Cấp quyền":

- Sử dụng Access Modifier `private`: Tất cả các biến (fields) trong Class nên được để ở chế độ `private`. Điều này ngăn chặn việc truy cập hoặc chỉnh sửa trực tiếp từ bên ngoài.

2. Cách triển khai thực tế trong Java

Để thực hiện đóng gói chuẩn chỉnh trong Java, chúng ta thường tuân theo quy trình "Che giấu - Cấp quyền":

1. **Sử dụng Access Modifier** `private` : Tất cả các biến (fields) trong Class nên được để ở chế độ `private` . Điều này ngăn chặn việc truy cập hoặc chỉnh sửa trực tiếp từ bên ngoài Class.
2. **Cung cấp các phương thức** `public` (**Getter và Setter**):
 - **Getter**: Cho phép bên ngoài "đọc" giá trị một cách an toàn.
 - **Setter**: Cho phép bên ngoài "ghi" giá trị, nhưng quan trọng nhất là ta có thể **kiểm soát (validate)** dữ liệu trước khi lưu vào biến.

Ví dụ cấu trúc chuẩn:

Java



```
public class User {  
    private String username; // Không ai có thể sửa trực tiếp biến này  
  
    // Getter  
    public String getUsername() {  
        return username;  
    }  
  
    // Setter với logic kiểm tra  
    public void setUsername(String username) {  
        if (username != null && username.length() > 5) {  
            this.username = username;  
        } else {  

```

3. Hậu quả nếu KHÔNG dùng tính đóng gói

Nếu em để mọi thứ là `public` (không đóng gói), hệ thống của em sẽ cực kỳ mong manh.

Ví dụ: Class `BankAccount` (Tài khoản ngân hàng)

Java

```
public class BankAccount {
    public double balance; // Rất nguy hiểm vì để public
}
```

Hậu quả:

Giả sử một lập trình viên khác trong team (hoặc kẻ xấu) vô tình viết code như sau:

Java

```
BankAccount account = new BankAccount();
account.balance = -1000000; // Sửa trực tiếp số dư thành số âm
```

Do biến `balance` là `public`, hệ thống sẽ chấp nhận con số `-1,000,000` này mà không có bất kỳ rào cản nào. Kết quả là ngân hàng bị lỗ, logic kinh doanh bị phá vỡ hoàn toàn.

Nếu em dùng **Encapsulation**, em sẽ để `balance` là `private` và trong hàm `setBalance()`, em sẽ thêm dòng check: `if (amount > 0)`. Khi đó, không ai có thể làm sai lệch dữ liệu của em được nữa.

Lời khuyên từ Anh: Hãy luôn bắt đầu với `private` cho các thuộc tính. Chỉ khi thực sự cần thiết mới cung cấp `public getter/setter`. Code sạch và an toàn bắt đầu từ những thói quen nhỏ này.

Tác vụ 2: Giải thích khái niệm phức tạp (Deadlock trong Hệ điều hành)

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
Cơ bản	Giải thích hiện tượng	AI liệt kê các khái niệm về tiến trình (process), tài nguyên (resource) đang chờ đợi lẫn nhau. Ngôn từ rất hàn lâm, mang đậm chất sách giáo khoa, người mới học đọc sẽ cảm thấy bối

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
	Deadlock là gì.	rồi. Hiện tượng Deadlock (Bế tắc) trong hệ điều hành là một trạng thái mà trong đó tiến trình (process) đang chờ đợi lẫn nhau để giải phóng tài nguyên (resource), không có tiến trình nào có thể tiếp tục thực thi. Cụ thể, hệ thống xảy ra tình trạng đang giữ khóa tài nguyên X và yêu cầu tài nguyên Y, trong khi tiến trình B đang giữ tài nguyên Y và yêu cầu tài nguyên X. Trạng thái chờ đợi vòng tròn này khiến toàn bộ tiến trình liên quan bị treo vĩnh viễn nếu không có sự can thiệp từ hệ điều hành. Các tài nguyên bị deadlock thường là CPU, bộ nhớ chính, hoặc các thiết bị ngoại vi độc quyền.
Cải tiến	Hãy giải thích khái niệm Deadlock (Bế tắc) trong môn Hệ điều hành. Lấy một ví dụ minh họa đơn giản sao cho một người không học IT cũng có thể hiểu được.	AI đã đổi giọng văn thân thiện hơn. Ví dụ được đưa ra thường là hai người cùng muốn dùng chung một thiết bị. Dễ hiểu hơn mức cơ bản nhưng cấu trúc bài viết vẫn còn khá lộn xộn. Chào bạn! Hiện tượng Bế tắc (hay còn gọi là Deadlock) trong máy tính xảy ra khi tiến trình bị kẹt cứng lại và không thể chạy tiếp được nữa. Nguyên nhân chính là do "giành giật" đồ đạc của nhau mà không bên nào chịu nhường. Để dễ hiểu hơn, bạn hãy tưởng tượng có hai người thợ mộc là anh A và anh B. Anh A đang cầm cây búa và cần cái đinh để đóng mộc. Anh B thì lại đang cầm cây búa. Anh A nhất quyết không bỏ búa xuống nếu chưa lấy được đinh, còn anh B cũng đinh chờ lấy được búa. Thế là cả hai cứ đứng nhìn nhau mãi mãi mà không ai làm việc cả! Trong hệ điều hành cũng vậy, hai phần mềm (giống như hai người thợ) cùng cần những thiết bị (như búa và đinh) mà bên kia đang giữ. Khi hệ thống không thể giải quyết, máy tính sẽ rơi vào trạng thái Deadlock và treo luôn ứng dụng đó.

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
Nâng cao	Áp dụng Kỹ thuật Tư duy từng bước (Chain-of-Thought) & Ẩn dụ (Analogy): Hãy giải thích Deadlock theo đúng 4 bước sau: Bước 1: Bắt đầu bằng ví dụ ẩn dụ về 4 chiếc ô tô ở một ngã tư đường giao nhau cùng lúc.	Hiệu quả tối đa. Nhờ ép buộc tư duy theo luồng, AI tạo ra một bài giảng hoàn hảo. Hình ảnh 4 chiếc ô tô kẹt cứng ở ngã tư giúp não bộ chốt chặt khái niệm. Sau đó, AI dẫn dắt rất mượt mà sang 4 điều kiện kỹ thuật phức tạp của hệ thống máy tính.

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
	Bước 2: Chỉ ra sự tương đồng: Ô tô là gì? Ngã tư là gì trong máy tính? Bước 3: Nêu 4 điều kiện cần thiết để xảy ra Deadlock. Bước 4: Tóm tắt lại bằng một câu khẩu quyết dễ nhớ.	

Chào bạn, rất vui được hỗ trợ một sinh viên CNTT như bạn làm rõ khái niệm này. Deadlock (Bế tắc) là một vấn đề kinh điển trong Hệ điều hành và Lập trình đa luồng.

Dưới đây là lời giải thích chi tiết theo 4 bước mà bạn yêu cầu:

Bước 1: Ví dụ ẩn dụ về 4 chiếc ô tô

Hãy tưởng tượng một ngã tư đường phố không có đèn giao thông. Cùng một lúc, có **4 chiếc ô tô** đi từ 4 hướng (Đông, Tây, Nam, Bắc) cùng tiến vào giữa ngã tư.

- Chiếc xe hướng Bắc muốn đi thẳng nhưng bị chiếc xe hướng Tây cản đường.
- Chiếc xe hướng Tây muốn đi thẳng nhưng bị chiếc xe hướng Nam chặn lại.
- Chiếc xe hướng Nam muốn đi thẳng nhưng lại vướng chiếc xe hướng Đông.
- Chiếc xe hướng Đông muốn đi thẳng nhưng bị chiếc xe hướng Bắc chắn lối.

Kết quả là cả 4 chiếc xe đều dừng khựng lại. Không ai có thể tiến thêm một mét nào, và cũng không ai chịu lùi lại để nhường đường. Đó chính là một trạng thái "tê liệt" hoàn toàn.

Bước 2: Sự tương đồng trong hệ thống máy tính

Từ ví dụ trên, chúng ta có thể ánh xạ sang các thành phần trong máy tính như sau:

- **Ô tô = Tiến trình (Process) hoặc Luồng (Thread):** Đây là các thực thể đang hoạt động và cần thực hiện nhiệm vụ của mình (đi qua ngã tư).
- **Đoạn đường/Ngã tư = Tài nguyên (Resource):** Đây là những thứ mà tiến trình cần để hoàn thành công việc. Tài nguyên có thể là phần cứng (CPU, máy in, ổ đĩa) hoặc phần mềm (biến dùng chung, file, Mutex/Semaphore).

- **Đoạn đường/Ngã tư = Tài nguyên (Resource):** Đây là những thứ mà tiến trình cần để hoàn thành công việc. Tài nguyên có thể là phần cứng (CPU, máy in, ổ đĩa) hoặc phần mềm (biến dùng chung, file, Mutex/Semaphore).
 - **Hành động lái xe vào ngã tư = Chiếm giữ tài nguyên:** Khi một chiếc xe đã nằm trên một phần đường, chiếc xe khác không thể đi vào phần đường đó được.
-

Bước 3: 4 điều kiện cần thiết để xảy ra Deadlock

Để một vụ tắc nghẽn trở thành Deadlock thực sự, phải có đủ 4 điều kiện sau (thường gọi là điều kiện Coffman):

1. **Loại trừ tương hỗ (Mutual Exclusion):** Tài nguyên không thể chia sẻ. Tại một thời điểm, chỉ có một tiến trình được sử dụng tài nguyên. (Giống như một khoảng đường tại ngã tư chỉ đủ cho một chiếc xe đứng).
 2. **Giữ và Đợi (Hold and Wait):** Một tiến trình đang giữ ít nhất một tài nguyên và đang đợi thêm tài nguyên mới mà tiến trình khác đang nắm giữ. (Xe đã vào ngã tư một nửa, giữ chỗ đó và đợi xe bên cạnh nhường đường để đi tiếp).
 3. **Không thể bị thu hồi (No Preemption):** Tài nguyên không thể bị "cướp" từ tiến trình đang giữ nó; chỉ khi tiến trình đó xong việc mới tự nguyện nhả ra. (Bạn không thể nhắc bóng chiếc xe của người khác ra khỏi đường để lấy lối đi).
 4. **Đợi vòng tròn (Circular Wait):** Tồn tại một chuỗi các tiến trình $\{P_0, P_1, \dots, P_n\}$ sao cho P_0 đợi tài nguyên của P_1 , P_1 đợi P_2 , ..., và P_n lại quay về đợi tài nguyên của P_0 . (Tạo thành một cái vòng khép kín như ví dụ 4 chiếc xe ở Bước 1).
-

thành một cái vòng khép kín như ví dụ 4 chiếc xe ở Bước 1).

Bước 4: Khẩu quyết dễ nhớ

Để dễ dàng nắm lòng bản chất của Deadlock, bạn có thể nhớ câu sau:

"Độc chiếm, giữ chờ, không nhả, đợi vòng."

- **Độc chiếm:** Mutual Exclusion.
- **Giữ chờ:** Hold and Wait.
- **Không nhả:** No Preemption.
- **Đợi vòng:** Circular Wait.

Hy vọng cách trình bày này giúp bạn nắm chắc kiến thức để áp dụng vào các bài thi hoặc khi lập trình thực tế!

Tác vụ 3: Tạo bộ câu hỏi ôn tập (Nhận diện Phishing)

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
Cơ bản	Tạo vài câu hỏi trắc nghiệm về tấn công lừa đảo qua mạng.	AI tự sinh ra 3 câu hỏi với nội dung quá hiển nhiên (ví dụ: "Có nên đưa mật khẩu cho người lạ không?"). Không có giá trị thực tế để ôn tập hoặc nâng cao kỹ năng. Câu 1: Tấn công lừa đảo qua mạng (Phishing) thường nhằm mục đích gì? A. Giúp máy tính chạy nhanh hơn B. Lấy cắp thông tin cá nhân, tài khoản và mật khẩu C. Tặng quà miễn phí cho người dùng D. Quảng cáo phần mềm diệt virus Câu 2: Nếu bạn nhận được một email từ người lạ yêu cầu cung cấp mã OTP, bạn nên làm gì? A. Cung cấp ngay để nhận thưởng B. Tuyệt đối không cung cấp mã OTP cho bất kỳ ai C. Gửi mã nhưng đổi mật khẩu sau D. Đưa cả tên đăng nhập và mật khẩu Câu 3: Bạn nên làm gì nếu thấy một đường link lạ gửi qua tin nhắn Facebook của mình? A. Bấm vào xem ngay lập tức B. Chia sẻ cho bạn bè cùng xem C. Không bấm vào và xóa tin nhắn D. Hỏi lại người gửi xem đó là gì

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
Cải tiến	Tạo một bộ 5 câu hỏi trắc nghiệm tình huống về cách nhận diện email lừa đảo (Phishing). Yêu cầu cung cấp 4 đáp án A, B, C, D và có đáp án đúng ở cuối bài.	Đã ra đúng một bài kiểm tra nghiêm túc. Các tình huống cụ thể hơn. Tuy nhiên, chỉ có đáp án đúng mà không giải thích tại sao các đáp án khác lại sai.

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
		Câu 3: Bạn nhận được email thông báo trúng thưởng iPhone 15 từ một nhân hà bạn trả phí ship 50.000 VNĐ qua thẻ tín dụng để nhận quà. Bạn nên làm gì? A. Nhập số thẻ tín dụng vì 50.000 VNĐ là số tiền nhỏ. B. Tìm kiếm thông tin chương trình trúng thưởng trên website chính thức của n C. Gọi điện cho ngân hàng hỏi xem có nên chuyển tiền không. D. Gửi lại số tài khoản ngân hàng thay vì thẻ tín dụng. Câu 4: Khi rê chuột (hover) qua một đường link trong email (không nhấp chuột) hiển thị góc dưới màn hình khác hoàn toàn với đoạn text hiển thị trong email. Đ là gì? A. Đây là tính năng rút gọn link an toàn của email. B. Hệ thống mạng đang bị lỗi hiển thị. C. Đây là dấu hiệu lừa đảo, đường link sẽ dẫn bạn đến một trang web khác với r D. Đường link đang được nâng cấp bảo mật. Câu 5: Một email có vẻ được gửi từ sếp của bạn (đúng tên, đúng chữ ký), yêu c khẩn cấp 10 thẻ cào điện thoại và gửi mã qua email vì sếp đang họp không tiện động chuẩn nhất là: A. Mua ngay để thể hiện sự nhiệt tình với sếp. B. Gọi điện thoại trực tiếp cho sếp để xác nhận lại yêu cầu. C. Nhắn tin qua Skype hỏi sếp loại mạng nào. D. Trả lời email xin sếp ứng trước tiền. Câu 1: Bạn nhận được một email từ "AdminIT@gmail.com" thông báo tài khoản công ty sẽ bị khóa trong 24 giờ tới nếu không xác minh bằng cách nhấp vào link đính kèm. Hành nào là đúng nhất? A. Nhấp vào link và điền thông tin để tránh bị khóa tài khoản. B. Trả lời email hỏi lại xem có chắc chắn là tài khoản bị khóa không. C. Bỏ qua email và báo cáo cho bộ phận IT thực sự của công ty qua kênh nội bộ. D. Chuyển tiếp email cho đồng nghiệp để cảnh báo. Câu 2: Dấu hiệu nào sau đây trong email báo cáo thuế KHÔNG phải là dấu hiệu của Phi A. Email bắt đầu bằng "Kính gửi quý khách hàng" thay vì tên thật của bạn. B. Địa chỉ email người gửi có tên miền là @https://www.google.com/search?q=tax-gov-r update.com. C. Email yêu cầu tải xuống một file .zip chứa "bản kê khai chi tiết". D. Email được gửi từ địa chỉ chính thức của cơ quan thuế và không chứa link yêu cầu nh khẩu.

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
Nâng cao	Kỹ thuật Đưa ví dụ mẫu (Few-shot Prompting): Đóng vai một chuyên gia An toàn thông tin. Tạo 3 câu hỏi tình huống tinh vi về Phishing phân loại theo độ khó: Dễ, Trung bình, Khó. Tuân thủ nghiêm ngặt định dạng sau: [Tình huống]: Mô tả một email hoặc tin	Mang tính ứng dụng cực cao. Việc cung cấp định dạng chuẩn giúp bài kiểm tra trông rất chuyên nghiệp. Phần [Gợi ý tư duy] buộc người đọc phải dừng lại suy nghĩ thay vì xem ngay kết quả. Câu hỏi mức độ "Khó" thực sự đưa ra các thủ đoạn tinh vi, rất hữu ích cho việc học tập.

Mức độ	Nội dung Prompt	Phân tích Kết quả đầu ra (Output Analysis)
	nhấn cụ thể. [Câu hỏi]: Điểm đáng ngờ nhất trong tình huống này là gì? [Gợi ý tư duy]: Hướng dẫn cách phân tích tình huống. [Đáp án chi tiết]: Chỉ rõ thủ đoạn của hacker.	

1. Cấp độ: Dễ (Mạo danh thương hiệu & Thúc giục tâm lý)

[Tình huống]: Bạn nhận được một email từ địa chỉ `support@netflix.com` (chú ý kỹ: chữ I trong từ "netflix" là chữ i viết hoa). Tiêu đề email là: "[Cảnh báo] Phương thức thanh toán của bạn bị từ chối". Email thông báo rằng nếu bạn không cập nhật lại thông tin thẻ tín dụng trong vòng 6 giờ tới, tài khoản của bạn sẽ bị xóa vĩnh viễn và mất toàn bộ lịch sử xem phim.

[Câu hỏi]: Điểm đáng ngờ nhất trong tình huống này là gì?

[Gợi ý tư duy]: Hãy quan sát thật kỹ các ký tự cấu thành nên địa chỉ email người gửi và phân tích tính chất của sự đe dọa trong nội dung.

[Đáp án chi tiết]: * **Thủ đoạn:** Hacker sử dụng kỹ thuật **Typosquatting** (hoặc Homograph attack) – thay thế ký tự 'l' (L thường) bằng 'I' (i hoa) để đánh lừa thị giác người dùng nhìn lướt qua.

- **Yếu tố tâm lý:** Sử dụng **Urgency** (Sự khẩn cấp) với thời hạn 6 giờ và hình phạt nặng (xóa tài khoản) nhằm ép nạn nhân hoảng sợ mà thực hiện hành vi theo bản năng, bỏ qua các bước kiểm tra an toàn.

2. Cấp độ: Trung bình (Chiếm đoạt thông tin xác thực qua tệp đính kèm)

[Tình huống]: Bạn nhận được email từ bộ phận Nhân sự của công ty với tiêu đề: "**Cập nhật danh sách thưởng và lộ trình tăng lương năm 2026**". Email đính kèm một tệp tin có tên `Chinh_sach_luong_2026.pdf`. Khi bạn mở tệp này bằng trình duyệt web, trang web hiện ra một nội dung bị làm mờ và một cửa sổ pop-up chuyên nghiệp yêu cầu: "*Vui lòng đăng nhập tài khoản Microsoft 365 của công ty để xác thực quyền truy cập tệp tin bảo mật này*".

[Câu hỏi]: Điểm đáng ngờ nhất trong tình huống này là gì?

[Gợi ý tư duy]: Hãy phân tích quy trình vận hành của các dịch vụ lưu trữ đám mây (như OneDrive, SharePoint). Việc yêu cầu đăng nhập xảy ra ở bước nào là hợp lý?

[Đáp án chi tiết]: * **Thủ đoạn:** Đây là kỹ thuật **Credential Harvesting** (Thu thập thông tin xác thực). Trong thực tế, nếu một tệp tin được bảo mật bởi hệ thống công ty, bạn sẽ được yêu cầu đăng nhập *trước* khi có thể tải hoặc mở tệp trên trình duyệt.

- Việc một tệp PDF đã mở ra thành công nhưng lại hiện pop-up yêu cầu đăng nhập bên trong nội dung là một dấu hiệu giả mạo rõ rệt. Hacker nhúng một liên kết ẩn dưới dạng giao diện đăng nhập giả để lấy tên đăng nhập và mật khẩu của nhân viên.
-

3. Cấp độ: Khó (Tấn công chiếm quyền điều khiển luồng hội thoại - Thread Hijacking)

[Tình huống]: Bạn đang trao đổi email với một đối tác lâu năm về một dự án cung cấp thiết bị. Đột nhiên, ngay trong luồng email đang trao đổi (phần Reply), đối tác gửi một email mới với đúng địa chỉ email cũ: *"Chào bạn, do phía ngân hàng của chúng tôi đang bảo trì định kỳ, bạn vui lòng cập nhật lại thông tin tài khoản nhận thanh toán cho hợp đồng lần này theo file đính kèm nhé. Xin lỗi vì sự bất tiện này"*. File đính kèm là một tệp `.docx` chứa thông tin tài khoản mới. Cách hành văn của đối tác có vẻ trang trọng hơn và thiếu đi vài câu chào hỏi thân mật như mọi khi.

[Câu hỏi]: Điểm đáng ngờ nhất trong tình huống này là gì?

[Gợi ý tư duy]: Khi các yếu tố kỹ thuật (địa chỉ email, tiêu đề, luồng hội thoại) đều có vẻ hoàn hảo, hãy tập trung vào sự thay đổi đột ngột trong "quy trình nghiệp vụ" và "phong cách hành văn".

[Đáp án chi tiết]: * **Thủ đoạn:** Đây là hình thức tấn công **BEC (Business Email Compromise)** thông qua **Email Thread Hijacking**. Hacker đã chiếm quyền điều khiển tài khoản email của đối tác (hoặc máy chủ email của họ) từ trước đó, âm thầm đọc các hội thoại để chờ thời cơ.

- Dấu hiệu cảnh báo nằm ở việc **thay đổi thông tin tài khoản thanh toán bất ngờ** qua email – một điều tối kỵ trong giao dịch thương mại. Ngoài ra, việc thay đổi đột ngột thói quen ngôn ngữ (Linguistic deviation) là manh mối duy nhất cho thấy người đang ngồi sau bàn phím không phải là đối tác quen thuộc của bạn.

Hãy luôn ghi nhớ nguyên tắc: **"Dừng lại - Suy nghĩ - Xác minh"** trước khi thực hiện bất kỳ yêu cầu nhạy cảm nào qua internet. Chúc bạn luôn giữ vững được sự tỉnh táo trước các thủ đoạn phishing tinh vi!

IV. Phân tích lý do tại sao một số Prompt hiệu quả hơn

Qua quá trình thực nghiệm trực tiếp và đối chiếu minh chứng, sự vượt trội của các prompt Nâng cao được lý giải qua các nguyên lý sau:

1. **Thiết lập vai trò (Role-playing):** Khi không được giao vai, AI mặc định sử dụng giọng văn bách khoa toàn thư chung chung. Khi được gán vai "Senior Developer" hay "Chuyên gia bảo mật", AI buộc phải kích hoạt kho từ vựng chuyên ngành sâu hơn, thay đổi cách hành văn trở nên thực tế và mang tính hướng dẫn.
2. **Kỹ thuật Chain-of-Thought (Tư duy từng bước):** AI thường có xu hướng "nhảy cóc" để đưa ra ngay kết luận. Việc ép AI phải trình bày theo trình tự (Bước 1, Bước 2...) giống như việc tạo ra một đường ray, giúp

luồng thông tin không bị lan man, logic chặt chẽ từ trực giác đến lý thuyết cốt lõi.

3. **Kỹ thuật Few-shot (Cung cấp mẫu định dạng):** Giải thích bằng lời cho AI về cấu trúc văn bản nhiều khi không hiệu quả bằng việc đưa ra sẵn các thẻ đánh dấu cụ thể. Nó giới hạn không gian sáng tạo tự do của AI, bắt buộc AI tuân thủ nghiêm ngặt định dạng đầu ra của người học.

V. Tổng hợp nguyên tắc và mẹo viết Prompt hiệu quả

Đề khai thác tối đa sức mạnh của AI trong học tập, dựa trên các bài test trên, người học nên xây dựng câu lệnh theo nguyên tắc **"4 chữ C"**:

1. **Context (Bối cảnh):** Cung cấp bức tranh toàn cảnh. (Ví dụ: "Tôi là sinh viên đang cần ôn tập môn Hệ điều hành để thi cuối kỳ").
2. **Character (Vai trò):** Chỉ định tư cách của AI. ("Hãy đóng vai một kỹ sư CNTT giàu kinh nghiệm, giải thích dễ hiểu").
3. **Constraints (Ràng buộc):** Đưa ra các giới hạn cụ thể để tránh thông tin rác. ("Chỉ dùng danh sách gạch đầu dòng", "Không dùng thuật ngữ quá hàn lâm").
4. **Clarity in Format (Định dạng rõ ràng):** Sử dụng các ví dụ mẫu (Few-shot) để định hình chính xác cấu trúc đầu ra mong muốn thay vì để AI tự đoán cách trình bày.